

Informe Final

Predicción de Cancelaciones de Reservas Hoteleras

Sistema automático de clasificación binaria

Máster en Inteligencia Artificial, Cloud Computing y DevOps

Módulo de Machine Learning y Deep Learning · PontIA.tech

Autores: David Baos y Juan Martínez Fraile

Repositorio: github.com/jumarfradev/practica-final-ml

Índice

Índice.....	2
1. Resumen ejecutivo	4
2. Roles del equipo	5
3. Justificación del problema y del conjunto de datos	6
3.1. El problema de negocio.....	6
3.2. El conjunto de datos.....	6
3.3. Elección de la métrica principal.....	7
4. Análisis exploratorio de datos (EDA)	7
4.1. Calidad de los datos y valores imposibles	7
4.2. Filas duplicadas. Una decisión deliberadas	8
4.3. Valores nulos. El caso de agent y company	8
4.3. Hallazgo contraintuitivo. El tipo de depósito	9
4.4. Otras variables predictivas	9
5. Diseño del sistema	10
5.1. Arquitectura modular	10
5.2. El pipeline de preprocesamiento	11
5.3. Eliminación del data leakage.....	11
6. Modelado y resultados	12
6.1. Comparativa de modelos.....	12
6.2. Modelo ganador y su optimización	12
6.3. Evaluación visual.....	13

7. Productivización y extensiones (bonus)	13
7.1. Automatización del flujo: trainer.py	13
7.2. Registro de experimentos con MLflow	13
7.3. API REST con FastAPI	13
7.4. Interfaz visual con Streamlit	14
7.5. Interpretabilidad con SHAP	14
7.6. Análisis del umbral óptimo por coste de negocio	15
7.7. Experimento de balanceo de clases	15
8. Calidad del código y prácticas de desarrollo	16
9. Reflexión crítica. Limitaciones y mejoras	17
9.1. Limitaciones	17
9.2. Mejoras futuras	17
10. Conclusiones	18

1. Resumen ejecutivo

Este proyecto implementa un sistema automático de Machine Learning para predecir si una reserva hotelera será cancelada. Se trata de un problema de clasificación binaria sobre el dataset `hotel_bookings` (119.390 reservas, 32 variables originales), cuya variable objetivo `is_canceled` presenta un desbalanceo moderado: aproximadamente un 63 % de reservas no canceladas frente a un 37 % canceladas.

El sistema entrena y compara ocho configuraciones de modelos (cinco algoritmos base y tres versiones optimizadas), selecciona el mejor según la métrica ROC-AUC y lo persiste para su uso en inferencia. El modelo ganador es un `RandomForest` optimizado con un ROC-AUC de 0,9581 sobre el conjunto de test.

Más allá de los requisitos mínimos, el proyecto incorpora una arquitectura de software modular, registro de experimentos con `MLflow`, una API REST con `FastAPI` para servir el modelo, una interfaz visual con `Streamlit`, interpretabilidad con `SHAP`, un análisis del umbral de decisión óptimo según coste de negocio, y un experimento que justifica empíricamente la decisión de no aplicar técnicas de balanceo.

Resultado principal

El modelo `RandomForest` optimizado alcanza un ROC-AUC de 0,9581 y un F1 de 0,8384, superando con holgura los valores de referencia del enunciado (~0,94). El sistema completo es ejecutable de extremo a extremo mediante un único script (`trainer.py`) y expone el modelo vía API e interfaz web.

2. Roles del equipo

La práctica se ha desarrollado en pareja, combinando trabajo conceptual conjunto y ejecución técnica. A lo largo del proyecto se mantuvieron reuniones de planteamiento teórico en las que se discutió cómo abordar la práctica: el diseño de la arquitectura, el enfoque de desarrollo y las decisiones de ejecución.

El reparto de responsabilidades fue el siguiente:

Integrante	Rol	Contribución
David Baos	Aportación teórica y conceptual	Contribución en el planteamiento teórico de la arquitectura y el desarrollo de la práctica: aportación de ideas de desarrollo y de ejecución, y propuestas para optimizar el enfoque práctico, trabajadas en las reuniones conjuntas.
Juan Martínez Fraile	Ejecución práctica e implementación	Desarrollo y ejecución práctica completa del proyecto: implementación del pipeline, los modelos, la evaluación, la API, la interfaz y la documentación. Responsable de la materialización del código en el repositorio.

Nota sobre la evaluación

El historial de commits del repositorio refleja la ejecución práctica, realizada por Juan Martínez Fraile. La contribución de David Baos se centró en el plano conceptual y de diseño (reuniones de planteamiento, aportación de ideas de arquitectura, desarrollo y optimización del enfoque), que por su naturaleza no queda registrada en commits.

3. Justificación del problema y del conjunto de datos

3.1. El problema de negocio

Las cancelaciones de reservas son uno de los principales problemas operativos de la industria hotelera. Una cancelación no anticipada implica una habitación que queda vacía sin margen para revenderse, con la consiguiente pérdida de ingresos. Anticipar la probabilidad de cancelación permite al hotel tomar decisiones proactivas: ajustar precios dinámicamente, aplicar políticas de overbooking informadas, u ofrecer incentivos de retención a las reservas de mayor riesgo.

Se trata de un caso de uso real y económicamente relevante, lo que lo hace idóneo para el objetivo del módulo: resolver un problema práctico de clasificación binaria de principio a fin.

3.2. El conjunto de datos

El dataset `hotel_bookings` contiene 119.390 reservas con 32 variables que describen tres dimensiones: características de la reserva (antelación, fechas, duración, precio), información del cliente (país, tipo, histórico previo) y el canal de contratación (segmento de mercado, agencia).

La variable objetivo, `is_canceled`, es binaria: 1 si la reserva se canceló, 0 si se completó. Su distribución es de aproximadamente 63 % / 37 %, un desbalanceo moderado que condiciona la elección de la métrica de evaluación (véase la sección 5).

3.3. Elección de la métrica principal

Se elige ROC-AUC como métrica principal, con F1 como métrica secundaria. La justificación es la naturaleza desbalanceada del problema:

- La accuracy sería engañosa: un modelo trivial que prediga siempre la clase mayoritaria (no cancela) alcanzaría ya un 63 % de acierto sin aprender nada útil.
- ROC-AUC mide la capacidad del modelo de ordenar las reservas por riesgo de cancelación, de forma independiente del umbral de decisión, y es robusta frente al desbalanceo. Es la métrica idónea cuando interesa priorizar reservas por riesgo.
- F1 (secundaria) equilibra precisión y exhaustividad (recall) sobre la clase positiva (cancelación), que es la de interés de negocio. Sirve de contraste al fijar un umbral operativo.

4. Análisis exploratorio de datos (EDA)

El EDA fundamenta todas las decisiones de preprocesamiento y modelado. Se resumen los hallazgos más relevantes y, en particular, varias singularidades del dataset que tienen impacto directo en el modelo.

4.1. Calidad de los datos y valores imposibles

Se detectaron registros con valores físicamente imposibles, atribuibles a errores de captura: precios por noche negativos ($adr < 0$) y reservas con más de diez adultos ($adults > 10$). Estas filas se eliminaron por tratarse de errores, no de valores atípicos legítimos. En cambio, los valores extremos pero plausibles ($lead_time$ muy alto, adr elevado pero positivo) se conservaron, ya que aportan información real al modelo.

4.2. Filas duplicadas. Una decisión deliberada

El análisis detectó 31.994 filas exactamente duplicadas, un 26,8 % del dataset. Es una proporción muy alta, por lo que se investigaron antes de tomar ninguna decisión. **Se concluyó conservarlas, no eliminarlas, por una razón metodológica, el dataset no dispone de un identificador único de reserva.** Sin él, no es posible distinguir un duplicado por error de registro de dos reservas reales distintas que coinciden en todos los campos disponibles, algo perfectamente posible en un hotel con alto volumen (dos clientes del mismo país, mismo tipo de habitación, misma antelación y mismas fechas). **Eliminar una cuarta parte de los datos asumiendo que todas son errores habría podido sesgar el modelo descartando reservas legítimas.** Se reconoce que eliminarlas es una alternativa defendible; la decisión de conservarlas se tomó de forma consciente y priorizando no perder información potencialmente válida.

4.3. Valores nulos. El caso de agent y company

Las variables agent y company presentan numerosos nulos. El análisis concluyó que se trata de nulos MNAR (Missing Not At Random): la ausencia ES información. Un nulo en agent indica una reserva directa (sin agencia); un nulo en company, una reserva no corporativa. Por ello, antes de imputar, se crearon dos variables binarias (tiene_agent, tiene_company) que capturan esa señal.

4.4. Hallazgo contraintuitivo. El tipo de depósito

Rareza destacada del dataset

Las reservas con depósito «Non Refund» (no reembolsable) presentan una tasa de cancelación cercana al 99 %, muy superior a las «No Deposit». Esto es contraintuitivo: cabría esperar que quien paga por adelantado sin reembolso no cancele.

La interpretación más plausible no es que los clientes cancelen reservas ya pagadas, sino que esta categoría refleja un artefacto de cómo se registran ciertas reservas en el sistema de gestión: reservas especulativas o en bloque de agencias que rara vez se materializan, o no-shows que acaban marcados como cancelaciones. Es un ejemplo claro de correlación sin causalidad directa: el modelo aprende que «Non Refund» predice cancelación, y acierta, pero el motivo subyacente requiere interpretación de negocio. Este matiz se retoma en la reflexión crítica (sección 9).

4.4. Otras variables predictivas

- `lead_time` (antelación de la reserva): a mayor antelación, mayor probabilidad de cancelación. Más tiempo implica más margen para cambios de planes.
- `market_segment`: las reservas a través de agencias online (Online TA) cancelan más que las directas o corporativas.
- `total_of_special_requests`: actúa como indicador de compromiso; más peticiones especiales se asocian a menor probabilidad de cancelar.
- `country`: existen diferencias notables de comportamiento según el mercado de origen.

5. Diseño del sistema

El sistema se ha construido como un proyecto de software modular, no como un único notebook. La lógica se reparte en módulos especializados dentro de `src/`, y un script orquestador (`trainer.py`) ejecuta el flujo completo de extremo a extremo.

5.1. Arquitectura modular

Módulo	Responsabilidad
<code>src/data_loader.py</code>	Carga del dataset, limpieza, creación de flags, reducción top-N de categóricas, construcción del ColumnTransformer y función orquestadora <code>preparar_datos()</code> .
<code>src/models.py</code>	Entrenamiento de los cinco modelos base y utilidades de MLflow.
<code>src/optimization.py</code>	Optimización de hiperparámetros (RandomizedSearchCV y búsqueda de arquitecturas).
<code>src/evaluation.py</code>	Generación de las gráficas de evaluación (curvas ROC, matriz de confusión, importancia de variables, comparativa).
<code>src/metrics_io.py</code>	Persistencia de las métricas de cada modelo en formato JSON.
<code>src/predictor.py</code>	Inferencia sobre datos crudos: reproduce la cadena de preprocesamiento y predice.
<code>src/api.py</code>	API REST con FastAPI que sirve el modelo (bonus).
<code>trainer.py</code>	Script principal que orquesta todo el flujo y persiste el modelo ganador.

5.2. El pipeline de preprocesamiento

El preprocesamiento se implementa con un ColumnTransformer de scikit-learn que aplica, a cada grupo de columnas, la transformación adecuada:

- Numéricas asimétricas (lead_time, adr): imputación por mediana, transformación logarítmica ($\log_{1p}$) y estandarización. La transformación logarítmica corrige la fuerte asimetría detectada en el EDA.
- Numéricas normales: imputación por mediana y estandarización.
- Categóricas de baja cardinalidad: imputación por moda y OneHotEncoder.
- Categóricas de alta cardinalidad (country, agent): reducción top-N + «Other» (se conservan las 30 más frecuentes) antes del OneHotEncoder, para evitar cientos de columnas casi vacías.
- Variables binarias (flags): pasan sin transformar.

El reparto train/test es estratificado (stratify=y, random_state=42) para preservar la proporción 63/37 en ambos conjuntos. El ColumnTransformer se ajusta únicamente sobre train y se aplica a test, evitando fuga de información.

5.3. Eliminación del data leakage

Decisión crítica

Se eliminaron tres variables que constituyen data leakage: reservation_status, reservation_status_date y assigned_room_type. Estas variables contienen información que solo se conoce DESPUÉS de que la reserva se complete o cancele, por lo que no estarían disponibles en el momento real de la predicción. Mantenerlas habría inflado artificialmente las métricas y producido un modelo inútil en producción.

6. Modelado y resultados

Se entrenaron los cinco algoritmos exigidos por el enunciado y, adicionalmente, tres versiones optimizadas, sumando ocho configuraciones comparables bajo la misma métrica.

6.1. Comparativa de modelos

Modelo	ROC-AUC	F1	
RandomForest OPT	0,9581	0,8384	GANADOR
XGBoost OPT	0,9577	0,8423	
XGBoost	0,9519	0,8332	
RedNeuronal OPT	0,9512	0,8312	
RedNeuronal	0,9492	0,8282	
RandomForest	0,9485	0,8088	
DecisionTree	0,9285	0,7854	
LogisticRegression	0,9102	0,7570	

6.2. Modelo ganador y su optimización

El modelo ganador es el RandomForest optimizado (ROC-AUC 0,9581). Sus hiperparámetros, hallados mediante RandomizedSearchCV, son: n_estimators=300, max_depth=25, min_samples_split=5, min_samples_leaf=1, max_features=sqrt.

Es relevante destacar que XGBoost optimizado queda a solo cuatro diezmilésimas (0,9577) y de hecho supera al ganador en F1 (0,8423 frente a 0,8384). La elección del RandomForest, por tanto, es ajustada y se sostiene en la métrica principal fijada a priori (ROC-AUC); ambos modelos son defendibles y la diferencia práctica es mínima.

6.3. Evaluación visual

Se generaron cuatro visualizaciones, guardadas en resultados/figuras/: curvas ROC comparativas de los ocho modelos, matriz de confusión del modelo ganador con etiquetas legibles, gráfico de importancia de variables (`feature_importances_`) y gráfico de barras de ROC-AUC. Las curvas ROC confirman visualmente la jerarquía: los modelos optimizados se sitúan más cerca de la esquina superior izquierda, y la regresión logística, al ser lineal, queda por debajo.

7. Productivización y extensiones (bonus)

El proyecto va más allá de los requisitos mínimos con varias extensiones orientadas a productivizar el modelo y enriquecer el análisis.

7.1. Automatización del flujo: `trainer.py`

El script `trainer.py` ejecuta el flujo completo: carga, preprocesamiento, entrenamiento de los ocho modelos, selección del mejor por ROC-AUC y persistencia. El modelo ganador se guarda como un paquete (`best_model.pkl`) que contiene no solo el modelo, sino el preprocesador ajustado, las categorías top-N congeladas y los metadatos, de modo que la inferencia puede partir de datos crudos.

7.2. Registro de experimentos con MLflow

Cada entrenamiento registra en MLflow sus parámetros, métricas y el modelo, lo que permite comparar experimentos de forma trazable a través de su interfaz web local.

7.3. API REST con FastAPI

Se expone el modelo mediante una API REST con cuatro endpoints: `/health` (estado), `/model_info` (metadatos), `/predict` (predicción individual) y `/predict_batch` (lote). La API recibe reservas en formato crudo y reproduce internamente todo el preprocesamiento.

7.4. Interfaz visual con Streamlit

Una aplicación Streamlit ofrece una interfaz gráfica que consume la propia API por HTTP. No duplica la lógica de predicción: actúa como cliente del backend, lo que demuestra una arquitectura desacoplada frontend/backend como la de un sistema real.

7.5. Interpretabilidad con SHAP

Además de `feature_importances_` (importancia global), se incorpora SHAP para explicar predicciones individuales. El endpoint `/predict_explain` devuelve, para cada reserva, las variables que más empujaron la predicción y en qué dirección. Esto permite responder no solo «cuánta probabilidad de cancelar», sino «por qué».

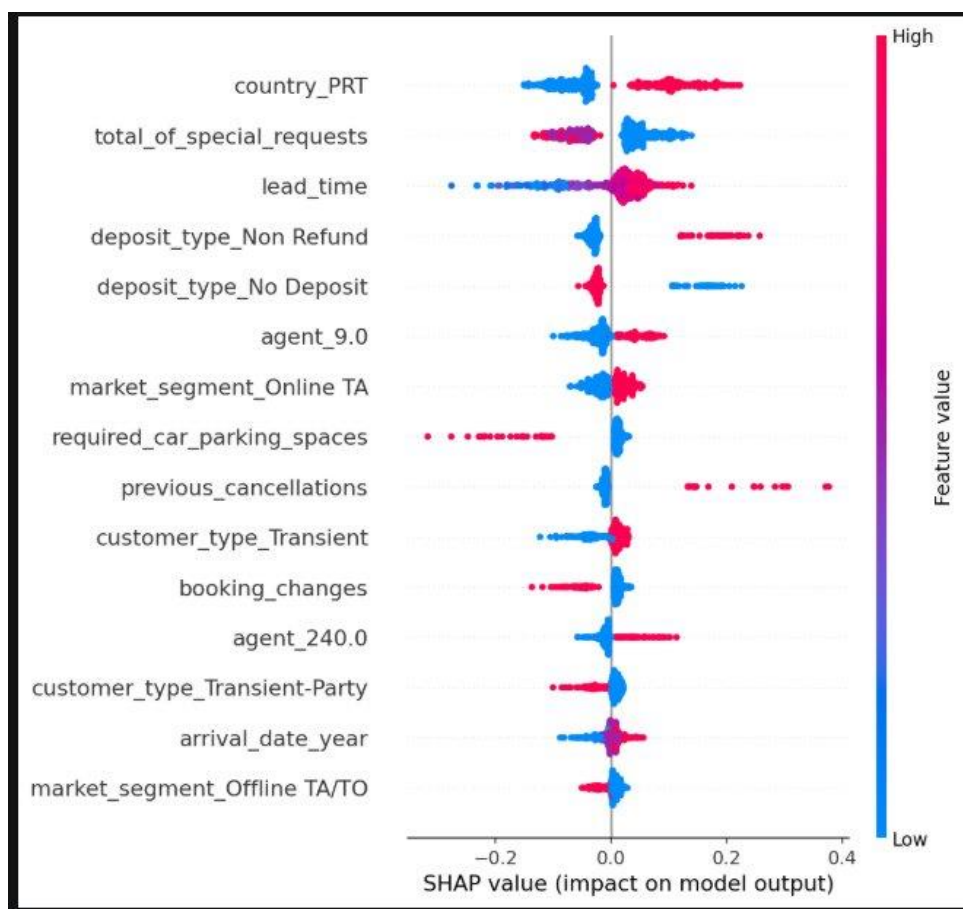


Figura: resumen SHAP de las variables más influyentes del modelo. Cada punto es una reserva; a la derecha empuja hacia la cancelación, a la izquierda hacia la no cancelación. El color indica el valor de la variable (rojo alto, azul bajo).

El análisis SHAP del modelo confirma las relaciones esperadas y revela los predictores más potentes. **Entre las variables de mayor impacto destacan:**

- El país de origen (en este dataset de hoteles portugueses, las reservas de Portugal concentran muchas cancelaciones)
- El número de peticiones especiales (efecto protector: más peticiones, menos cancelación)
- El lead_time (a más antelación, más riesgo)
- El tipo de depósito (Non Refund empuja con fuerza hacia la cancelación).

Resultan especialmente reseñables dos variables muy predictivas:

- previous_cancellations (quien ya canceló antes tiende a repetir)
- required_car_parking_spaces (reservar aparcamiento, indicador de un viaje planificado, reduce notablemente la probabilidad de cancelación).

7.6. Análisis del umbral óptimo por coste de negocio

El umbral por defecto de 0,5 trata ambos errores por igual, pero en el negocio hotelero no cuestan lo mismo. Un falso negativo (no anticipar una cancelación) implica una noche perdida, mientras que un falso positivo (acción de retención innecesaria) tiene un coste menor. Bajo el supuesto razonado de que un falso negativo cuesta del orden de cinco veces más que un falso positivo, se recorren todos los umbrales y se identifica el que minimiza el coste total de negocio, que resulta ser inferior a 0,5. Este análisis conecta el modelo estadístico con la decisión operativa real.

7.7. Experimento de balanceo de clases

Se evaluó empíricamente el efecto de balancear las clases (class_weight y SMOTE) frente a no hacerlo. El experimento confirma que, con un desbalanceo moderado (63/37) y un modelo que ya alcanza un F1 de 0,84, el balanceo apenas modifica el ROC-AUC y solo desplaza el equilibrio entre exhaustividad y precisión, sin mejora neta. Se decide, por tanto, no aplicar balanceo y entrenar sobre la distribución real, una decisión respaldada por evidencia y no por omisión.

8. Calidad del código y prácticas de desarrollo

El proyecto sigue buenas prácticas de ingeniería de software:

- Formateo y linting automáticos con Black y Ruff, ejecutados como hooks de pre-commit en cada commit.
- Integración continua con GitHub Actions, que verifica la calidad en cada push.
- Protección de la rama main mediante ruleset: los cambios entran exclusivamente a través de Pull Request con la verificación de calidad superada.
- Flujo de trabajo basado en ramas (una rama por funcionalidad), documentado en CONTRIBUTING.md.
- Docstrings en las funciones y nombres descriptivos, siguiendo PEP 8.
- Entorno reproducible gestionado con uv y Python 3.11, con las dependencias fijadas en requirements.txt.

9. Reflexión crítica: limitaciones y mejoras

9.1. Limitaciones

- **Correlación frente a causalidad:** el modelo capta patrones predictivos (como el del depósito «Non Refund») cuyo origen real requiere interpretación de negocio. Un patrón predictivo no implica una relación causal accionable.
- **Top-N calculado sobre todo el dataset:** las categorías frecuentes de country y agent se determinaron sobre el conjunto completo y no solo sobre train. Al no usar la variable objetivo, el impacto es despreciable, pero metodológicamente lo ideal sería calcularlo únicamente en train.
- **Validación:** la comparación de modelos se basa en una única partición train/test estratificada. Una validación cruzada completa daría estimaciones de métrica más robustas (la optimización sí usa validación cruzada internamente).
- **Supuesto de coste fijo:** el análisis de umbral asume una proporción de coste 5:1 entre falso negativo y falso positivo. El valor real depende del hotel concreto y debería calibrarse con datos de negocio.

9.2. Mejoras futuras

- Validación cruzada estratificada en la comparativa final de modelos.
- Codificación cíclica (seno/coseno) de las variables temporales (mes, semana) para capturar mejor la estacionalidad.
- Monitorización de la deriva de datos (data drift) del modelo en producción.
- Calibración de probabilidades para que el umbral de coste sea aún más fiable.
- Despliegue de la API y la interfaz en un entorno cloud con contenedores.

10. Conclusiones

El proyecto cumple y supera los objetivos del enunciado. Se ha construido un sistema automático, modular y reproducible que entrena, compara y selecciona modelos de clasificación binaria, alcanzando un **ROC-AUC de 0,9581** con el RandomForest optimizado, por encima de los valores de referencia.

Más allá del rendimiento, el valor del proyecto reside en el rigor de las decisiones: la eliminación del **data leakage**, la elección justificada de la métrica, el análisis crítico de las rarezas del dataset, la decisión razonada de no balancear y la conexión del modelo con el coste de negocio real mediante el análisis de umbral. **La productivización (trainer, API, interfaz, interpretabilidad)** acerca el trabajo a un escenario profesional real, que es el objetivo último del módulo.